

Inventory Review: Gaps and Bugs

Abode Admin asset detail (land inventory) · reviewed 1 October 2026 · frontend branch
abode-inventory-tracking, backend abode-be-v2 staging at commit f7952cc

What this covers. The asset detail screens were reviewed against the approved HTML design (Asset Detail Planning V2) and against the backend source and the live staging API. This document records every bug found, every change made, and every gap that is still open. Part A is the frontend; Part B is the backend.

Headline findings.

1. Three finished tabs (Performance, Customers, Updates) were hidden because the frontend wrongly recorded their endpoints as missing. All three exist in source and on live staging. They are now shown.
2. Every numeric input in the inventory screens had a bug that made a cleared field show its old value while the form held nothing, which blocked saving. Fixed in the shared input.
3. The backend's sqm ledger totals count every sized purchase twice. The frontend now works around it; the backend fix is still needed.
4. Three frontend features call endpoints that do not exist, and six figures drawn in the design have no backend source yet.

Verification status. Frontend changes pass the TypeScript check and lint, a 9-case script covers the new calculations, and every asset tab compiles on the running dev server. The changes have not been end-to-end tested in a browser against real data by the reviewer, and the Playwright suite was not re-run. Nothing has been committed.

Part A: Frontend

Repository: `abode-fe-admin`. File paths are relative to it.

A1. Bugs found and fixed

#	Bug	Cause	Fix
F1	Cleared number fields refill themselves and block saving. Deleting the last digit made the original number reappear. The field then showed a figure with a 'required' error under it and the form could not be saved. Seen in the Land account drawer; affects every numeric input in the inventory screens.	The shared input sends 'empty' to the form as undefined. react-hook-form answers an undefined field with the value it had when it mounted, and the input wrote that echo back into the box while the form itself stayed empty.	<code>components/detail/NumberInput.tsx</code> now remembers its mount value and, while the user has the field cleared, treats that echo as 'still empty'. One fix covers all users of the component: land account, roads and services, offers, sizes, plans, blocks and plots, boundary, costs, allocation rules, selling charges, create asset.
F2	Performance, Customers and Updates tabs hidden. Outside mock mode the tabs were removed from the navigation and their pages showed a 'backend gap' notice.	The gap document and code comments stated the endpoints returned 404 and had no backend module. Both claims were wrong (see section B7).	Gate removed from the three pages and from <code>AssetDetailNav.tsx</code> . Response schemas were compared with the backend DTOs field by field first; all three match.
F3	Ledger figures would double-count. Anything reading the sqm inventory response's totals block shows inflated capacity, selling and sold figures.	Backend bug B1.	New helpers <code>productPositions()</code> and <code>ledgerTotals()</code> in <code>schemas/sqm-inventory.schema.ts</code> add up pool rows only. The header bar, Management summary, Product position and event readiness all use them.
F5	Selling charges did not work against the real API. The panel always showed 'No selling charges approved yet', the history drawer was always empty, and saving failed. No error was shown for any of it.	The backend changed this contract on 28 September (commit 0f042ef): GET now returns the version in force plus scheduled versions, rows carry <code>is_latest</code> instead of <code>is_current</code> , and PUT requires <code>expected_version</code> . The frontend still expected the old shapes, so every response failed validation, and the screens treated a failed read as 'nothing set up'.	Schema, hooks, panel, editor, history sheet and mock route rewritten to the current contract. The panel now shows the version in force and any scheduled versions; the editor sends <code>expected_version</code> and handles the 409; a failed read now shows an error instead of an empty state. New script <code>scripts/selling-charges-mock-qa.ts</code> (5 cases).
F4	Gap document was wrong and contradicted the code. <code>docs/ASSET-LAND-INVENTORY-BACKEND-GAPS.md</code> sections 1.5 to 1.7 and the plots/summary note.	Written from an earlier, incorrect check.	Sections corrected; new section 6 added for the findings in this review.

A2. Updates made

Area	Change
Tab navigation	Order now follows the design: Overview, Offers, Blocks & Plots, Performance, Costs, Customers, Updates. Site Setup is not in the design and was moved to the end rather than removed.
Header bar	For estates on the live sqm ledger: 'X of Y saleable sqm available' with sold, selling and available segments. Estates still on legacy units keep the unit bar.
Attention banner (new)	Lists unclassified land, land account warnings, incomplete cost items and cost entries with no amount. Hidden when there is nothing to fix.

Area	Change
Asset details	Four-column layout with Estate layout and Gallery, as designed. It is now the single editable panel: its Edit form also carries visibility, sales cap, images and documents, saved as one request.
Removed panels	Availability, Roads & services and Images and documents no longer appear as separate Overview panels. Their fields moved into the Asset details edit form (visibility, sales cap, images, documents) and the Land account drawer (roads and services rows). Value history is a line inside Asset details.
Land account card	Restyled to the design with exact figures (for example 315,230 sqm · 73.7%) and the 'Product pools assigned' line.
Management summary (new)	The design's four questions: land accounted for, commercially available, next allocation event, profitability.
Product position (new)	One row per product with SQM, Units and Value lenses. Units are the ledger's per-size rows divided by plot size; Value is those units at each size's current outright land price.
Allocation event readiness (new)	Capacity, reserved, remaining and already allocated for the next upcoming event. Replaces the old Allocation events list card.
Financial position (new)	Revenue, total cost, net profit and margin, realised cash, and missing inputs.
Land account drawer	Wider, with the live reconciliation pinned at the top, product pools in a two-column grid with the input beside the name, and roads and services as one line per row.
Offers tab	New 'Offer land pools' table (assigned, configured, unconfigured, unit capacity, status) with Add offer in its header. Offer cards follow the design: status, Edit offer, Add size; size headers show configured units and sqm with an Edit button. Plan rows gained the Price version column, filled from the estate's current selling-charges version. The selling charges panel moved off the page into a 'Price versions' side sheet.
Blocks & Plots tab	New five-figure summary strip from GET .../plots/summary (previously unused). Block cards follow the design (name, status, plots / available / allocated) and open the block's plot editor when clicked; Delete block moved into that editor. Plot inventory is now the design's table (plot, block, size, product, status, customer plan, allocated date) with a Manage plots button. Removed from the tab: the sqm inventory panel, the two status matrices, the reconciliation panel and the ground confirmation badge (none is in the design; the last three have no backend).
Performance tab	Rebuilt to the design: date filter with 'Figures as of', a five-figure summary strip, defaults and terminations, customer health with collection efficiency, the payment plan table with the design's ten columns, then 'Profitability by product' with the design's columns (forecast revenue, direct cost, shared cost, gross profit, margin) and View calculation (moved here from the Costs tab) and a provisional-profit notice driven by the backend's own warnings. All tables scroll sideways on small screens instead of switching to a separate card layout.
Estimated profit columns removed	The payment plan table used to show gross profit, allocated opex, net contribution and margin per size and tenor. Those were produced in the frontend by splitting the estate's total cost across rows by share of revenue, which the backend never calculated. They are gone from that table. Real profit is shown by product, and by product, size and tenor in its own table from profitability/matrix (see 'Performance: profit by payment plan' below).
Plan table balances	Default balance, terminated value and terminated balance are shown as a second line inside the Default value and Terminated cells, keeping the design's ten columns.
Direct and shared cost	The design's Direct cost and Shared cost columns are derived from profitability/drill-down: a cost charged to one product is direct, a cost split across several is shared. New helper costSplitByProduct() with a test script. The 'Gross profit' column carries the backend's net profit figure, because operating costs are already inside the two cost columns.

Area	Change
Profit calculation drawer	Rebuilt as 'How the profit is worked out': the sum first (forecast revenue minus direct cost minus shared cost equals gross profit, with margin), then a notice listing costs that are not counted and why, revenue by product (opening to sizes and payment plans), direct costs, shared costs with each product's slice and the split rule in plain words, and cost per square metre. It uses the same words as the Profitability by product table and shows product and rule names instead of internal codes. Covered by scripts/profitability-split-qa.ts (6 cases).
Costs tab (page)	Rebuilt to the design: a five-figure summary strip (approved budget, committed, incurred, paid, forecast remaining); a notice listing what is still unknown, from the backend's coverage report; the Asset costs table with one row per cost item and the design's columns (scope, budget, committed, incurred, paid, remaining forecast, status); 'How shared cost is distributed'; 'Recent cost changes'; and a Cost history sheet. The tab is renamed 'Costs'. The profitability card and coverage panel were removed from it (profit is on Performance and Overview; coverage feeds the notice and the Status column). Figures are added up in schemas/cost-ledger.schema.ts, covered by scripts/cost-ledger-qa.ts (7 cases).
Costs: how the figures are fetched	Because of backend issue B10, the page reads every cost record's detail (one request each) and adds the stages up per cost item and for the estate. It covers the newest 100 records and says so when an estate has more. The per-record reads share a cache with the record detail sheet, so opening a record costs nothing extra.
Costs: unknown is not zero	A stage with no approved entry shows a dash, never 0. Entries awaiting approval are counted as 'N pending' in the Status column and are in no figure. A cost item with nothing recorded is 'Cost missing' with 'Unknown' remaining forecast.
Costs: one-form Add cost	The design's single 'Add asset cost' form replaces the separate cost item dialog and cost record drawer. On save it makes up to three backend calls in order: create the cost item if it is new (with its split rule), create the record with its first entry, and approve that entry. 'Save draft' stops before the approval. If a later step fails, the form keeps what was saved and a retry repeats only the failed step, so no duplicate item is created. Rules and sequencing live in schemas/add-cost.schema.ts, covered by scripts/add-cost-qa.ts (11 cases, including a save that fails halfway and is retried).
Add cost: fields adapted from the design	'Forecast' is not offered as something to record (no forecast stage exists). 'Scope' is 'shared across products' or one product. 'Linked operational record' is left out: the backend sets a record's source itself and an admin entry is always manual. 'Evidence' uploads the file first and sends its link.
Costs: Revise modal	The design's Revise modal replaces the record detail sheet. It shows the record's budget, committed, incurred and paid figures and five choices: Correct details (edit an entry still awaiting approval), Create revision (replace the budget), Add commitment, Record invoice and Record payment, with an impact preview of the remaining forecast. Below is the record's entry list, where waiting entries are approved and approved ones reversed. Rules and sequencing live in schemas/revise-cost.schema.ts, covered by scripts/revise-cost-qa.ts (6 cases).
Revise: how a budget revision is saved	Budget entries add up and cannot be negative, so a revised budget cannot be one edit. The modal adds the revised budget, reverses the old approved one(s), then approves the new one. The order means a save that stops partway leaves the budget understated, never doubled, and pressing Save again finishes it without entering anything twice. The revision source is written at the start of the entry's note, since the backend has no field for it.
Revise: fields adapted from the design	'Revised remaining forecast' is not an input: it is worked out as budget less incurred and shown in the impact preview. The preview is arithmetic on the record's own figures; the backend's impact preview only covers split-rule changes. Profit impact is shown for an invoice only, because a budget does not affect profit.
Mock server: repeated entries at a stage	The mock refused a second manual entry at the same stage on a record. The real backend allows it (each manual entry gets its own key), so the mock now does too.

Area	Change
Customers tab	Rebuilt as the design's 'Customer land position' table: Customer, Product, Size, Purchase state, Land treatment, Allocation. Purchase state comes from the plan's status and default flag (Selling, Overdue, Defaulted, Suspended, Sold, Cancelled, Closed). Allocation is joined by plan id from the plot inventory. The figures the old twelve-column list showed (referrer, tenor, amount paid, balance, progress, next payment, date joined) are kept as a second line in the cell they belong to. Search, the subscriber filter, sorting and CSV export are unchanged. Covered by scripts/customer-land-position-qa.ts (7 cases).
Updates tab	Now opens on the design's 'Asset history': one timeline, newest first, merged in the browser from six real sources (land account versions, price versions, boundary versions, verified field work, cost entries, allocation events already held). Each line names the area it came from. If an area cannot be read (no permission, or its request failed) the panel says which, so a short timeline is not mistaken for a complete one. GET .../field-history, live but previously unused, is now consumed. The buyer announcements feed, a separate working feature the design has no screen for, stays reachable as a second view on the same tab. Covered by scripts/asset-history-qa.ts.
Site Setup: shapes re-compared	GET .../site-setup, GET .../boundary and PUT .../boundary were compared field by field with the current backend source. They match; nothing needed fixing. The tab, however, showed only the boundary and fencing. It now also shows what the same response already carried: boundary established, land cleared, plots parcelled and re-pegged, and repaired and remaining metres per side.
Site Setup: field work review	New. The tab lists the estate's field submissions (GET /admin/field-submissions?asset_id=) with a status filter, opening on work awaiting review. Each opens a sheet (GET /admin/field-submissions/:id) with the work details, evidence, receipts, plots, warnings and what verifying wrote. From there an admin can verify (acknowledging any warning, and optionally accepting a surveyor's proposed boundary), reject, correct the figures, reverse, or link plots to unmapped clearing. Only the actions the backend accepts in the submission's state are offered. Needs view_field_submissions and verify_field_submissions.
Site Setup: field team, events, costs	New panels. Field team: who covers the site (GET .../field-staff) with each worker's targets and score for a chosen month (GET .../field-performance). Allocation events on the ground: expected against confirmed customers, plans and sqm per event, and the accountable site manager, who can be assigned, changed or removed (/admin/field-allocation/*). Cost of field work: total, by category and entry by entry (GET .../field-costs).
Performance: profit by payment plan	GET .../profitability/matrix had a hook but no screen. It now has a table under 'Profitability by product': one row per product, size and tenor with the backend's own gross profit, allocated OPEX, net contribution and margin. It is closed until asked for, so the design's layout is what opens.
Overview: pitch pack	The Asset details panel now shows the estate's pitch pack (the PDF realtors download) and lets an admin upload, replace or remove it (PUT, DELETE .../pitch-pack). The asset schema did not read pitch_pack before.
Costs: cost groups, archive, remove	The Add cost form takes its finance groups from GET .../costs/catalogue, falling back to the built-in list. A cost record can be archived with a reason from its detail (PATCH .../costs/:id/archive). A cost item with nothing recorded on it can be removed from the table (DELETE .../costs/items/:id). Both hooks existed with no screen.
Blocks & Plots: block details, single plot	A block's description can be edited, and its label while it has no plots (PATCH /admin/blocks/:id, a hook with no screen until now; see B20 for why the label is restricted). Adding exactly one plot now uses POST /admin/blocks/:id/plots; more than one still uses the bulk route.

Area	Change
Mock server: field work	The mock Site Setup figures were invented fractions of the boundary. They are now worked out, as on the real backend, from the effects of verified submissions, so verifying, correcting or reversing a submission in mock mode moves fencing, clearing, costs and scores. Covered by scripts/field-operations-qa.ts (12 cases) and two new cases in scripts/profitability-split-qa.ts.
Sqm activation	The 'Activate sqm inventory' control moved from Blocks & Plots to a notice on the Overview, shown only while an estate is still on legacy units.
Mock server	The mock plot routes now honour the allocation and field_state filters the real backend supports, and serve plots/summary.
Tests	e2e/live-inventory.spec.ts and e2e/offers.spec.ts also updated, not re-run. scripts/live-inventory-mock-qa.ts gained a summary case (15 pass). New permanent script scripts/product-position-qa.ts (9 cases). e2e/land-configuration.spec.ts updated for the new layout but not re-run.

A3. Frontend work still open

Item	Detail
All seven design tabs are rebuilt	Overview, Offers, Blocks & Plots, Performance, Costs, Customers and Updates now follow the design. None has been checked in a browser against real data by the reviewer, and the Playwright suite has not been re-run since the changes.
Field staff accounts are not built	Inviting field workers, enabling and disabling them, assigning them to sites, and setting, publishing and restating their monthly targets (/admin/field-staff/*, /admin/field-scorecards/*) and the organisation-wide performance views (/admin/field-performance/*) have no screens. They span every estate and are staff administration, not part of one asset's inventory, so they were left out of this pass. One consequence on the asset page: an allocation event can only be given to a site manager already assigned to the estate, and that assignment cannot be made from this app yet.
Site Setup and the new panels are unverified in a browser	They compile and their logic is covered by scripts against the mock server, but none has been opened against real staging data.
Features that only work in mock mode	Offer configuration history, ground confirmation, and the inventory reconciliation panel with its two status matrices. They call endpoints that do not exist (B2) and stay hidden against the real API.
Value lens uses today's prices	It values land at the current price list, not at what each customer paid. A true sold value needs the backend to report value per status (B3).
Schemas re-verified against the current backend	F5 showed that a schema marked 'confirmed against the backend' can be out of date. Re-verified since, against the backend source: analytics, subscribers, estate updates, selling charges, plot inventory, sqm inventory, asset costs (items, records, entries, coverage, allocation rules), profitability (summary, drill-down, matrix), site setup and boundary, and the field endpoints added in the last pass. Checked by reading what changed in the backend since 24 September rather than field by field: the land account (the save now returns real ids for new rows; same shape), the asset record (two added fields, state and pitch_pack) and allocation events (one added field, seats_taken). Added fields do not break the frontend. Not re-compared at all: blocks and plots, and the offer, size and plan write responses.
Unused code removed	AssetAllocationEventsCard, LandUseTable, BackendGapNotice and the three per-section edit hooks were no longer mounted and have been deleted.

Part B: Backend

Repository: `abode-be-v2`, branch `staging`. Routes are under `/api/v1`.

B1. Bugs

#	Bug	Evidence	Suggested fix
B1	Sqm ledger totals double-count sized purchases. GET <code>/admin/assets/:assetId/sqm-inventory</code> returns totals that are too high for capacity, selling, sold and available.	Read in source, <code>sqm-inventory.service.ts: keysFor()</code> returns the pool row and the size row, so each movement is applied to both; <code>positionFor()</code> then sums every row.	Sum pool rows (<code>size_id: null</code>) only for the commercial totals.
B2	Resolved. Land configuration PUT returned <code>id: null</code> for newly created roads and services rows.	Fixed in commit <code>0f042ef</code> (28 Sep): the response is now recomputed from a fresh read. Confirmed by reading the diff.	None. The frontend's refetch workaround can be removed.
B3	Resolved. Selling charges PUT had no stale-write guard.	Fixed in commit <code>0f042ef</code> : <code>expected_version</code> is required and a stale save returns <code>409 SELLING_CHARGE_VERSION_CONFLICT</code> . Confirmed in source.	None on the backend. The frontend had not been updated for it (see F5).
B4	Size <code>configured_units</code> goes stale after an edit and leaks into GET <code>/admin/assets/:id</code> .	Previously documented from source and staging (section 5). Not re-checked in this review.	Serialise sizes through <code>SizedTo</code> , or keep the field in step on update.
B5	Subscribers aggregates block never reaches the client . The response interceptor rebuilds the envelope from data, message and meta only.	Recorded in the frontend schema header; not re-checked in this review.	Forward aggregates through the interceptor.
B6	Compiled JavaScript is committed under <code>src/</code> . 1,088 <code>.js</code> files are tracked beside their TypeScript sources.	<code>git ls-files "src/**/*.js"</code> after pulling staging.	Confirm whether this is intended; if not, remove and ignore them.

Also resolved in commit `0f042ef`: selling charges can now be approved for a future date (scheduled versions), which the earlier gap document listed as missing. GET `/admin/assets/:assetId/plots/summary` was added in the same commit.

B2. Endpoints the frontend calls that do not exist

Endpoint	Used by	Effect today
GET <code>/admin/assets/:assetId/offers/history</code>	Offers tab History button	Hidden outside mock mode.
GET, POST <code>/admin/plots/:plotId/ground-confirmation</code> POST <code>.../ground-confirmation/:id/verify</code>	Ground confirmation badge on plots	Hidden outside mock mode. A wanted capability: separate system allocation from confirmation on the ground.
GET <code>/admin/assets/:assetId/inventory-reconciliation</code>	Physical and commercial status matrices, reconciliation panel	All three hidden outside mock mode.

B3. Figures in the design with no backend source

Design element	What is missing	Frontend today
Product position: Event reserved column	An allocation event stores one reserved total, with no per-product split.	Column left out.
Product position: Value lens	The ledger reports one purchase value per row across all live plans, not one per status.	Estimated from today's outright prices; labelled as such.
Header bar: defaulted-retained segment	Defaulted sqm is reported as a subset of selling and sold, not as its own bucket of capacity.	Named in the caption, not drawn as a segment.
Financial position: cost paid, committed cost; Costs tab summary strip	No estate-level total per financial stage (budget, committed, incurred, paid).	Direct cost and allocated opex shown instead.
Asset details: State	The asset has a location string only, no state field.	Field left out.
Costs: totals by stage per cost item and for the estate	Approved amounts by stage are returned per cost record only. The repository already has the aggregate (eventsByStage: stage and cost item, with entry counts and unknown amounts) but no endpoint exposes it.	The frontend reads every record's detail (one request each, first 100 records) and adds them up. A GET <code>.../costs/summary</code> returning that aggregate would replace all of them.
Costs: Scope	A cost item has no scope field (whole asset, common-use land, section, boundary segment). It has the products it applies to or excludes, and each record's product.	Scope is described from those.
Costs: forecast and 'remaining forecast'	There is no forecast stage or figure. Stages are budget, committed, claimed, incurred, paid, reversal, adjustment.	Remaining forecast is shown as budget minus incurred; 'Unknown' with no budget.
Costs: one row, one record	The design treats each table row as a single record (AC-0041) with one budget, commitment, incurred and paid figure. The backend has cost items holding any number of records, each with staged entries that need approval.	Rows are cost items; an item with several records opens to list them.
Costs: revise with impact preview	Impact preview exists for allocation-rule changes only, not for a budget or forecast revision. There is no revision source or supporting-evidence upload on a revision.	Not built.
Performance: 'Land retained' and 'Land released'	The defaulting and terminated buckets carry customers, plans and naira only, no sqm.	Both shown as a dash.
Performance: profitability for a date range	The profitability endpoint takes an as-of date, not a range.	Profitability is always current; the footnote says so.
Customers: Land treatment	The subscribers row has no land treatment. When a plan is closed the admin chooses whether to free its land or keep it (free_inventory, keep_inventory_reason), and that choice is not returned.	'Retained' for a live plan; a dash for a cancelled or closed one.

Design element	What is missing	Frontend today
Customers: Allocation	The subscribers row carries no plot or allocation status, although the plan stores both (allocations, allocation_status).	Joined in the browser from GET <code>.../plots?allocation=allocated</code> , which needs a second permission (view_field_performance) and returns at most 200 plots per page; beyond that, or without the permission, the column shows a dash rather than 'Awaiting allocation'.
Customers: allocation event	The design shows the event a plot was allocated in ('Event Batch 1'). Neither the subscriber row nor the plot row names it.	Not shown.
Plot inventory: Product on available plots	A plot has no product of its own; the product is read from the plan it is allocated to.	Dash on every unallocated plot.
Plot inventory: Customer plan	The endpoint returns only payment_plan_id, not the customer's name or the allocation event.	Shortened plan id shown.
Summary strip: 'N plans' reserved for the next event	The upcoming event reports reserved sqm and capacity, not the number of plans selected.	Reserved sqm shown instead.
Block cards: 'Survey work reported' status	No per-block field work status is returned with blocks or their plots.	Status is derived from plot counts only.
Offers: price version per plan	Prices are versioned once per estate (selling charges). A plan's own land price is a plain editable field with no version or history.	Every plan row shows the estate's selling-charges version.
Updates tab: one asset history feed	History exists per area behind separate endpoints with separate permissions; nothing merges them into one timeline or pages it.	Merged in the browser from six reads; cost entries cost one request per record.
Asset history: events the design shows that nothing returns	A purchase closed and its land released; plans added to or removed from an allocation event; offer, size and plan edits; sqm ledger movements. The admin audit log records many of these but GET <code>/admin/audit-logs</code> filters by admin and action only, not by estate.	Absent from the timeline. An estate filter on the audit log, or a dedicated asset history endpoint, would supply them.

B4. Offers tab: price versioning

Found while rebuilding the Offers tab. The design shows a price version on every payment plan row (for example 'v4 - 12 Sep 2026') and no separate charges panel. The backend's pricing model does not support that as drawn.

#	Issue	Detail	What is needed
B7	Prices are versioned per estate, not per plan.	The only versioned price record is selling charges (GET, PUT <code>/admin/assets/:assetId/selling-charges</code>), one version for the whole estate. A plan's own land_price is a plain editable field: changing it creates no version and keeps no history. The frontend therefore shows the same estate-wide version on every plan row, and a dash until a version is in force.	Either version the plan price itself (a version and effective date on each plan, with history), or state that selling charges are the single source of buyer price and remove the editable plan price. Confirm which is intended.

#	Issue	Detail	What is needed
B8	Two sources of price with no link between them.	Selling charges can carry a land price line, and each plan also carries a land price. Nothing in the API says which one a buyer is charged, or keeps them in step. This review did not trace the purchase flow to confirm which is used.	Document which figure the purchase flow charges, and reject or reconcile saves that make the two disagree.
B9	Selling charges have no home in the design.	The module is real and needed (it is the price versioning), but the approved design has no panel for it on the Offers tab. The frontend moved it into a 'Price versions' side sheet, opened from the header or from any Price version cell.	Product decision, not a code defect: confirm the side sheet is acceptable or add the panel to the design.

Related frontend display choice. Land prices on plan rows are shown in full (for example N9,600,000) rather than the design's shortened form (N9.6m), because this is the price list admins edit from and the short form hides differences below the rounding. This is a frontend decision and can be switched on request; it needs nothing from the backend.

B5. Costs tab: what it needs from the backend

Found while rebuilding the Costs tab. The frontend schemas for cost items, cost records, entries, coverage and allocation rules were compared with the backend source and are current; nothing in this module changed after the frontend was written except selling charges. The issues below are about what the API offers, not about mismatched shapes.

#	Issue	Detail	What is needed
B10	No endpoint returns cost totals by stage.	Approved budget, committed, incurred and paid amounts are returned for one cost record at a time (GET /admin/assets/:assetId/costs/:obligationId, fields stages and recognised_cost). Nothing returns them per cost item or for the whole estate, which is what the design's summary strip and cost table show.	Add GET /admin/assets/:assetId/costs/summary: totals by stage for each cost item and for the estate.
B11	The query for B10 already exists but is not exposed.	AssetCostRepository.eventsByStage(assetId) groups approved entries by stage and cost item, with the entry count and the number of entries that have no amount. No service or controller calls it.	Expose it through the endpoint in B10. Little new backend work is needed.
B12	Cost of the workaround.	To fill the page today the frontend reads the record list and then every record's detail, one request per record, and adds them up. The list endpoint returns at most 100 records per page, so on an estate with more the figures cover the newest 100 only; the page states this when it happens.	B10 removes all of these requests and the 100-record limit. On the frontend it is a one-file change (hooks/use-cost-ledger.ts).
B13	The data model differs from the approved design.	The design draws each table row as a single record (for example 'AC-0041') with one budget, one commitment, one incurred and one paid figure, edited through a single Add or Revise form. The backend has three layers: a cost item, any number of records under it, and staged entries on each record that only count once approved.	Product and backend decision: either the design is updated to the three-layer model, or the API offers a one-step create (item, record and first entry together). The frontend currently shows one row per cost item, opening to its records.

#	Issue	Detail	What is needed
B16	BUG: reversing an incurred cost subtracts it twice.	<code>reverseEvent()</code> sets the original entry's status to 'reversed' and also writes an approved 'reversal' entry for the same amount. Every total reads approved entries only (<code>getObligation</code> , <code>approvedEvents</code> for coverage and profitability), so the original is no longer counted and the reversal then subtracts it again. A reversed 1,000,000 cost becomes -1,000,000, and profit is overstated by that amount. Confirmed by reading the source. The backend's unit tests ('takes a reversal back off the recognised cost', 'nets a reversal off the cost it reversed') keep the original as 'approved', which is not the state the real reversal leaves it in, so they pass.	Do one or the other, not both: either leave the original 'approved' and let the reversal entry net it off, or mark it 'reversed' and write no reversal entry. Add a test that calls <code>reverseEvent</code> and then reads the total. The frontend shows the backend's figure as returned; <code>scripts/revise-cost-qa.ts</code> has a case that pins the current behaviour.
B17	A cost record's own details cannot be changed.	There is no update call for a record. Title, description, vendor, reference, product and effective date are fixed at creation; the only write on the record itself is archive. The design's 'Correct details' therefore can only correct an entry that is still a draft.	Add <code>PATCH /admin/assets/:assetId/costs/:obligationId</code> for the record's descriptive fields.
B18	A budget cannot be revised in one step.	Stage amounts add up and must be zero or more, and an approved entry cannot be edited. Changing a budget from X to Y takes three calls: add Y, reverse X, approve Y. There is no revision source or 'revised from' link between the two entries.	A revise call that replaces the budget atomically and records the reason and source would match the design's 'Create revision'.
B15	Adding a cost takes up to three calls and can half-finish.	Creating a cost item, creating a record under it and approving its first entry are three separate requests with no transaction across them. If the second fails, an item exists with no record; if the third fails, the cost is saved but uncounted.	A single create that takes the item (or its id), the record and the first entry together, with an option to approve, would make the design's one form atomic. The frontend currently sequences the three and resumes from the failed step on retry.
B14	Unknown and pending amounts are not distinguishable in totals.	A stage with no approved entry is simply absent from stages, and entries awaiting approval are in no total. The estate-level coverage report gives one count of entries awaiting approval and one of entries without an amount, not per cost item.	In the B10 summary, return <code>null</code> (not 0) for a stage with no approved entry, and include per-item counts of pending entries and entries with no amount.

B6. Found while wiring the remaining endpoints

Found by reading the backend source for each endpoint before building against it. B19 to B22 are defects; B23 to B27 are gaps or rough edges.

#	Issue	Detail	What is needed
B19	BUG: archiving a cost record does not stop its cost counting.	<code>archiveObligation()</code> only sets the record's status to 'archived'. Its approved entries are untouched, and every total (record detail, coverage, profitability) reads approved entries without looking at the record's status. So an archived record still adds to cost and reduces profit. There is also no way to un-archive, and no check on what is being archived.	Either exclude archived records from totals, or refuse to archive a record that still has approved entries. The frontend's archive dialog says the approved amounts still count and should be reversed first.

#	Issue	Detail	What is needed
B20	BUG: renaming a block leaves its plots with the old label.	Each plot stores a copy of its block's label (<code>block_label</code>). <code>updateBlock()</code> changes the block only, so after a rename the plot list, plot inventory and field submission plots still show the old label. The update also has no duplicate-label check, unlike <code>create</code> .	Update <code>block_label</code> on the block's plots in the same operation, and apply the duplicate check. Until then the frontend only allows a label change while the block has no plots.
B21	BUG: correcting a parcelation submission cannot change its plots.	<code>correct()</code> accepts a new payload and re-checks it, but writes the new effects with the submission's old <code>plot_ids</code> and never updates <code>plot_ids</code> on the submission. A corrected plot list changes the count but not which plots are marked parcelled. Separately, the status 'corrected' is never set by any code path (a corrected submission stays 'verified'), although <code>fieldHistory()</code> filters on it.	Set <code>plot_ids</code> from the corrected payload and write the effects with them. Either set the 'corrected' status or remove it. The frontend offers only the amount for parcelation corrections.
B22	BUG: a verified fencing submission warns that it exceeds the boundary because it counts itself twice.	<code>getOne()</code> recomputes warnings on every read. For new fencing it adds the submission's own metres to the metres already verified on that side, which, once the submission is verified, already include it. A side fenced to exactly its approved length reads as over by the submission's own length.	Skip the submission's own effects when it is already verified, or return the warnings stored at verification for a reviewed submission.
B23	A worker with no targets is reported as scoring zero.	<code>GET .../field-performance</code> returns <code>total_score: 0</code> for a worker with no scorecard that month, and the site's <code>average_score</code> includes them. Zero there means 'nothing to score against', not 'scored nothing'.	Return <code>null</code> for a worker with no scorecards and leave them out of the average. The frontend shows a dash and recomputes the average over workers who have targets.
B24	The per-estate allocation list leaves out events with no owner.	<code>GET /admin/field-allocation/assets/:assetId</code> returns only events that have an accountable site manager, so an event cannot be found there in order to give it one.	Return every allocation event on the estate, with <code>owner: null</code> where there is none. The frontend reads the estate's events from the company events list and fetches each unowned one separately.
B25	The profit matrix gives a size's id but not its area.	<code>GET .../profitability/matrix</code> rows carry <code>size_id</code> only, although the calculation has <code>size_sqm</code> . A size later removed from the asset cannot be labelled.	Add <code>size_sqm</code> to each row. The frontend looks the area up in the asset's offers and shows 'Size not on this asset' when it cannot.
B26	Field cost categories are keys with no labels.	<code>GET .../field-costs</code> returns categories such as <code>site_works</code> with no display label, and they are not the cost groups of the Costs tab.	Return a label with each category, as the cost catalogue does. The frontend formats the key.
B27	Small inconsistencies.	The cost catalogue is the same for every estate but is served per asset (<code>/admin/assets/:assetId/costs/catalogue</code>). Removing a pitch pack is logged under the admin action 'remove-asset'. The pitch pack upload takes a link and a size the client reports, with no check that the link is a PDF of that size.	Low priority. Listed so they are not rediscovered.

B7. Live endpoints: what is used now, and what is not

All confirmed in source and in the live staging API description. Every endpoint scoped to one asset is now used by a screen. What remains unused is staff administration across all estates.

Endpoint	Status
GET /admin/assets/:id/analytics GET /admin/assets/:id/subscribers (+ /export) GET, POST, PATCH /admin/assets/:id/updates (+ /publish, /archive)	Used. Previously recorded as missing (F2).
GET /admin/assets/:assetId/plots/summary	Used: Blocks & Plots summary strip.
GET /admin/assets/:assetId/field-history	Used: Updates tab timeline.
GET /admin/assets/:assetId/field-costs GET /admin/assets/:assetId/field-performance GET /admin/assets/:assetId/field-staff	Used: Site Setup, 'Cost of field work' and 'Field team'.
GET /admin/field-submissions (+ /:id) POST .../:id/verify, reject, correct, reverse, link-plots	Used: Site Setup, 'Field work' review.
GET /admin/field-allocation/assets/:assetId GET /admin/field-allocation/events/:eventId PUT, DELETE .../events/:eventId/owner	Used: Site Setup, 'Allocation events on the ground'.
GET /admin/assets/:assetId/profitability/matrix	Used: Performance, 'Profit by payment plan'.
GET /admin/assets/:assetId/costs/catalogue PATCH .../costs/:obligationId/archive DELETE .../costs/items/:itemId	Used: Costs tab (Add cost groups, archive a record, remove an unused item).
POST /admin/blocks/:block_id/plots PATCH /admin/blocks/:block_id	Used: Blocks & Plots (one plot, block details).
PUT, DELETE /admin/assets/:id/pitch-pack	Used: Overview, Asset details.
/admin/field-staff/* (list, invite, enable, disable, assignments) /admin/field-scorecards/* /admin/field-performance/* (summary, staff, metrics, trend, blockers, source records)	Not used. Field staff accounts, monthly targets and organisation-wide performance. Not scoped to one asset; see A3.